Edge Detection

Week 5 — Session 1: Gradients & derivative operators

Practical Image Processing

Lecture 5.1 ▪ 60–70 min

Recap & today

- Wk3–4: smoothing/denoising with kernels — killing **high frequencies**.
- Today (Session 1): **edges** — where intensity changes sharply. We treat an edge as a large **derivative** and build the classic gradient operators (Sobel, Prewitt, Scharr, Laplacian/LoG).
- Next session: assemble these into the **Canny** detector.

DocScan milestone

Finding the page/meter boundary starts with edges. This week's output feeds the "find the page" step later in the course.

Roadmap — Session 1

What is an edge?

First derivative: gradient operators

Second derivative: Laplacian & LoG

Noise & why we smooth

In-class exercise

Practice notes

Wrap-up

What is an edge?

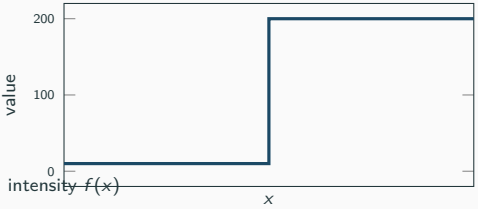
An edge = a rapid intensity change

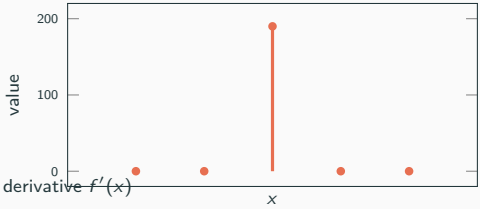
- Where the image jumps from dark to light (or vice versa) — object boundaries, text strokes.
- Mathematically, a large **gradient**: the derivative of intensity is high there.
- Edges are **high-frequency** (Week 3) — so we detect them with a high-pass kernel.

Math

Gradient $\nabla f = (f_x, f_y)$; magnitude $|\nabla f| = \sqrt{f_x^2 + f_y^2}$; direction $\theta = \text{atan2}(f_y, f_x)$.

A 1D edge and its derivative





Key idea

The edge is exactly where the derivative spikes. "Find edges" = "find large derivatives".

Four edge profiles you will meet



- **Step**: ideal boundary. **Ramp**: a blurred/soft edge (real optics).
- **Roof**/**line**: thin structures — a stroke of text, a wire, a meter needle.
- First derivative peaks at steps/ramps; a thin **line** shows up as a *pair* of opposite peaks.

First derivative: gradient operators

Derivatives as kernels

Sobel-x (vertical edges):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Sobel-y (horizontal edges):

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Each computes a *difference* across the pixel — a discrete derivative. Weights sum to 0, so a flat region gives 0, exactly as an edge kernel should.

Separable

Sobel-x = smoothing column $[1, 2, 1]^\top$ times differencing row $[-1, 0, 1]$: blur one axis, difference the other. That is why Sobel is a *denoised* derivative.

Worked example: Sobel-x on a patch

A 3×3 neighbourhood (a dark-to-light vertical edge):

10	10	200
10	10	200
10	10	200

$$
\begin{aligned}
g_x &= (-1)(10) + (1)(200) \\
&\quad + (-2)(10) + (2)(200) \\
&\quad + (-1)(10) + (1)(200) \\
&= 4(200 - 10) = \mathbf{760}
\end{aligned}
$$

A big positive response: a strong vertical edge, dark→light left to right.

✔ Good practice

Sign carries direction: a light→dark edge would give -760. Take $|g_x|$ for an "edge strength" map.

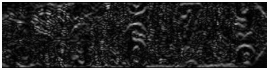
Sobel on the real meter



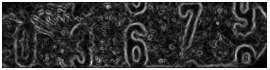
gray



∂x (vertical edges)



∂y (horizontal)



magnitude

x-gradient highlights vertical strokes; y-gradient horizontal ones; magnitude combines both.

Gradient magnitude & direction

- **Magnitude** $= \sqrt{g_x^2 + g_y^2}$: how strong the edge is (the bright edge map).
- **Direction** $= \text{atan2}(g_y, g_x)$: which way intensity increases; the edge itself runs *perpendicular* to this.
- Threshold the magnitude to get a binary edge map — but a fixed threshold is fragile.

✕ Pitfall

Gradients are noise-sensitive: noise creates strong fake gradients. Always smooth first (Week 3/4).

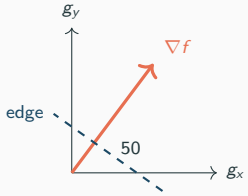
Worked example: magnitude & direction

Suppose at a pixel $g_x = 30$, $g_y = 40$.

$$|\nabla f| = \sqrt{30^2 + 40^2} = \sqrt{2500} = \mathbf{50}$$

$$\theta = \operatorname{atan2}(40, 30) \approx \mathbf{53}^\circ$$

Gradient points up-and-right; the edge line runs at $53^\circ - 90^\circ = -37^\circ$.



Key idea

Magnitude answers "how strong?"; direction answers "which way?" — Canny needs both.

Computing the gradient in OpenCV

```python
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)   # d/dx
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)   # d/dy
mag  = cv2.magnitude(gx, gy)                       # sqrt(gx^2 + gy^2)
ang  = cv2.phase(gx, gy, angleInDegrees=True)      # direction
mag8 = cv2.convertScaleAbs(mag)                    # back to 0..255 for display
```

✗ Pitfall

Use a *signed* type (`CV_64F`). On `uint8` the negative half of every edge is clipped to 0 — you lose one side of each edge.

The first-derivative family

Operator	x-kernel	Idea	Notes
Roberts	2×2 diagonal	diagonal diff	tiny, very noise-sensitive
Prewitt	$[-1, 0, 1]$, equal rows	simple gradient	no centre weighting
Sobel	$[-1, 0, 1]$, rows $1, 2, 1$	centre-weighted	the practical default
Scharr	rows $3, 10, 3$	optimized 3×3	better rotational accuracy

Prewitt-x: $\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$ **Scharr-x**: $\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$

Key idea

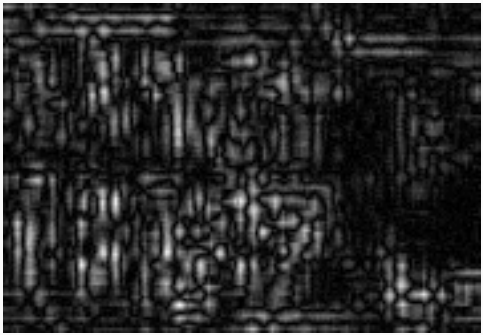
They all estimate the same derivative; the weights trade noise-rejection for accuracy.

Second derivative: Laplacian & LoG

Laplacian: the second derivative

$$\text{Laplacian} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

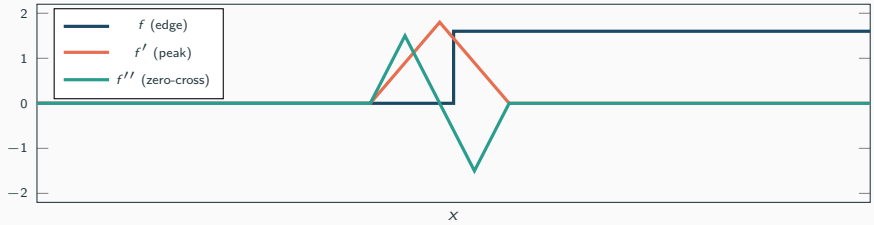
$\nabla^2 f = f_{xx} + f_{yy}$ — one kernel, sensitive to edges in *all* directions.



Laplacian of a plate

- Detects edges as **zero-crossings**: the 2nd derivative flips sign right at the edge.
- One kernel for every orientation — but even more noise-sensitive than the first derivative.

Zero-crossings: 1st vs 2nd derivative



- First derivative: **peak** at the edge. Second derivative: **zero-crossing** at the edge.
- Finding a zero-crossing localizes the edge precisely — but you must smooth first.

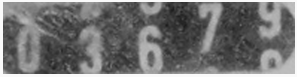
LoG: Laplacian of Gaussian

- Problem: the bare Laplacian screams at every noise pixel.
- Fix: **smooth with a Gaussian, then take the Laplacian** — the **LoG** operator.
- Because convolution is associative, this is *one* pre-built kernel (the "Mexican hat").

Math

LoG $= \nabla^2(G_\sigma * f) = (\nabla^2 G_\sigma) * f$. Bigger $\sigma \Rightarrow$ smoother, fewer/coarser edges. DoG (difference of two Gaussians) approximates it cheaply.

Laplacian on the meter vs plate



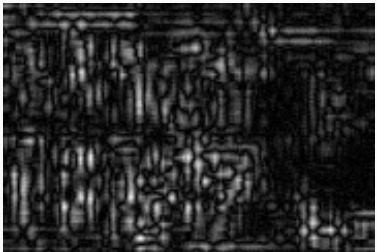
meter (gray)



meter Laplacian



plate (gray)



Noise & why we smooth

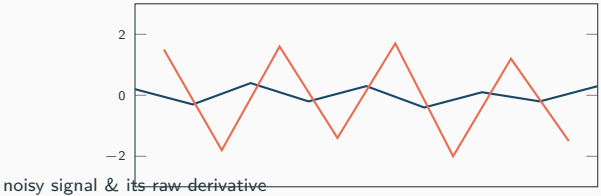
Why smooth before differentiating

- Differentiation **amplifies** high frequencies — including noise.
- So the derivative of a noisy signal is dominated by noise, not edges.
- Smooth first (Gaussian), then differentiate — the basis of Sobel/Canny/LoG.

Math

Handy identity: (blur then differentiate) = convolve with the *derivative of the Gaussian*, $\frac{d}{dx}(G_\sigma * f) = (\frac{d}{dx} G_\sigma) * f$ — one kernel does both.

Noise turns into fake edges



noisy signal & its raw derivative

- The flat signal has small wobbles (noise).
- Its derivative (coral) swings wildly — every wobble looks like an “edge”.
- A single Gaussian blur first flattens these before differentiating.

In-class exercise

In-class exercise (10 min)

Reason about a 1D signal

A row of pixels: `[10, 10, 10, 200, 200, 200]`.

1. Compute the first derivative as forward differences $f[i+1] - f[i]$. Where is it maximum?
2. Apply the 1D kernel $[-1, 0, 1]$ (a mini Sobel-x) centred on each interior pixel. What is the peak value, and at which pixel?
3. Now one pixel is noise: `[10,10,90,200,200,200]`. Recompute the differences — how big is the spurious response, and how would a blur reduce it?

Debrief

- Differences: `[0,0,190,0,0]` — the derivative spikes at the $10 \to 200$ jump. That spike *is* the edge.
- $[-1, 0, 1]$ centred at the pixel just left of the jump gives $200 - 10 = 190$: the peak lands on the edge.
- The noise pixel (90) makes differences `[0,80,110,0,0]`: a smaller, split response. A blur averages 90 back toward its neighbours before it becomes a false edge.

Key idea

An edge is where the derivative is large. Everything today is a way to compute and clean that derivative.

Practice notes

Common pitfalls

✗ Don't differentiate raw, noisy pixels

A bare Laplacian or Sobel on an unsmoothed photo returns a speckled mess. Blur first.

✗ Don't clip the sign

Storing a gradient in `uint8` clips negatives to 0 — half of every edge vanishes. Use a signed type, then take $|\cdot|$ for display.

Good practice

✔ Sensible defaults

Gaussian blur (3×3 or 5×5) → Sobel `ksize=3` → magnitude. Reach for Scharr when a 3×3 Sobel looks direction-biased.

✔ Match the tool to the job

First derivative (Sobel) for edge *strength/direction*; Laplacian/LoG when you want all-orientation response or precise zero-crossing locations.

Wrap-up

What you should now be able to do

- Explain an edge as a large gradient / a zero-crossing of the 2nd derivative.
- Compute Sobel-x/y by hand on a patch; get magnitude and direction.
- Compare Roberts/Prewitt/Sobel/Scharr and the Laplacian/LoG.
- Say why smoothing must come before differentiating.

Next session teaser

Session 2 — Canny & clean contours

A thresholded Sobel magnitude is thick, broken and noisy. Next time we chain **smooth** → **gradient** → **non-maximum suppression** → **hysteresis** into the **Canny** detector — thin, connected, tunable edges — and connect them to the DocScan page finder.

(No deliverable yet — bring your Sobel/Laplacian code from today.)

An edge is where the derivative is large.

Today: how to *compute* that derivative and keep noise out of it.